

PRACTICAL MANUAL ON JAVA



**ISBAT
UNIVERSITY**

A CHARTERED UNIVERSITY

**By
Kibwika N Bashir**

2025

**FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY
INTERNATIONAL BUSINESS SCIENCE AND TECHNOLOGY UNIVERSITY
Plot 11A, Rotary Avenue, Lugogo Bypass, Kololo, Kampala, Uganda**

List of Experiments

1	You are tasked with designing a system for a zoo that tracks the behavior of different animals. Each animal can make a sound, but the type of sound depends on the specific animal. Additionally, some animals can fly, while others cannot. You decide to use polymorphism to implement this system efficiently.
2	You are hired to design a payment processing system for an e-commerce application. The system should support multiple payment methods such as CreditCard , PayPal , and BankTransfer . Each payment method has its own way of processing a payment. The system should be flexible enough to allow the addition of new payment methods in the future without significant changes.
3	You are hired by a logistics company to develop a system to manage different types of vehicles. Each vehicle can calculate its maximum range based on its fuel capacity and fuel efficiency. The company has three types of vehicles: Car , Truck , and Motorcycle . Each type has unique characteristics for calculating the range.
4	A tech company wants to build a Payment Processing System that supports multiple payment methods such as Credit Card , Debit Card , and Digital Wallet . To ensure flexibility and security, the company has the following requirements:
5	A travel agency provides an online booking system for customers to reserve seats on flights. Due to increasing demand, they need to ensure their booking system can handle errors gracefully and enforce business rules.
6	A software company has developed an application that manages user accounts. The application has a feature to transfer funds between accounts. However, the system must handle several situations, including invalid input, insufficient balance, and maximum transfer limits.
7	You are tasked with creating a Student Registration System using Java Swing for the GUI and a database (e.g MS SQL Server) to store student information. The system should allow the user to input student details (Name, Age, Course, Email) and save them into the database. Additionally, the system should have a feature to display all registered students in a JTable.
8	You are tasked with creating a Hotel Reservation System using Java Swing for the user interface and a database (e.g., MS SQL Server) for storing reservation details. The system allows users to book a room and view existing reservations.
9	You are tasked with creating a Payroll Management System using Java Swing for the user interface and Microsoft SQL Server for the database. The system allows HR personnel to manage employee salary details, including adding new employee records, calculating net pay, and viewing all employee payroll records.

10	You are tasked with creating a Sales Inventory Management System using Java Swing for the user interface and Microsoft SQL Server for the database. The system will allow users to add products, update stock levels, record sales, and view inventory details
----	---

Questions 1

You are tasked with designing a system for a zoo that tracks the behavior of different animals. Each animal can make a sound, but the type of sound depends on the specific animal. Additionally, some animals can fly, while others cannot. You decide to use polymorphism to implement this system efficiently.

Requirements:

1. Create an abstract class `Animal` with the following:
 - A method `makeSound()` that will be overridden by subclasses.
 - A method `canFly()` that will be overridden to indicate whether the animal can fly.
2. Implement the following concrete subclasses:
 - **Dog:** Makes a sound "Woof" and cannot fly.
 - **Cat:** Makes a sound "Meow" and cannot fly.
 - **Bird:** Makes a sound "Tweet" and can fly.
3. Write a `Zoo` class to manage a list of animals. It should include:
 - A method `addAnimal(Animal animal)` to add an animal to the zoo.
 - A method `displaySounds()` to print the sounds of all animals in the zoo.
 - A method `displayFlyers()` to list all animals that can fly.
4. Write a main program to:
 - Create instances of `Dog`, `Cat`, and `Bird`.
 - Add them to the `Zoo`.
 - Call `displaySounds()` and `displayFlyers()`.

Solution

```
// Abstract class Animal

abstract class Animal {

    // Abstract method to make sound

    public abstract void makeSound();
```

```
// Abstract method to check if the animal can fly  
public abstract boolean canFly();  
}
```

```
// Concrete subclass: Dog  
class Dog extends Animal {  
    @Override  
    public void makeSound() {  
        System.out.println("Woof");  
    }  
  
    @Override  
    public boolean canFly() {  
        return false; // Dogs cannot fly  
    }  
}
```

```
// Concrete subclass: Cat  
class Cat extends Animal {  
    @Override  
    public void makeSound() {  
        System.out.println("Meow");  
    }  
  
    @Override  
    public boolean canFly() {  
        return false; // Cats cannot fly  
    }  
}
```

```
}
```

```
// Concrete subclass: Bird
```

```
class Bird extends Animal {
```

```
    @Override
```

```
    public void makeSound() {
```

```
        System.out.println("Tweet");
```

```
    }
```

```
    @Override
```

```
    public boolean canFly() {
```

```
        return true; // Birds can fly
```

```
    }
```

```
}
```

```
// Zoo class to manage animals
```

```
class Zoo {
```

```
    private List<Animal> animals;
```

```
// Constructor to initialize the list of animals
```

```
    public Zoo() {
```

```
        this.animals = new ArrayList<>();
```

```
    }
```

```
// Method to add an animal to the zoo
```

```
    public void addAnimal(Animal animal) {
```

```
        animals.add(animal);
```

```
    }
```

```
// Method to display sounds of all animals
public void displaySounds() {
    System.out.println("Animal Sounds:");
    for (Animal animal : animals) {
        animal.makeSound();
    }
}

// Method to display animals that can fly
public void displayFlyers() {
    System.out.println("\nAnimals that can fly:");
    for (Animal animal : animals) {
        if (animal.canFly()) {
            System.out.println(animal.getClass().getSimpleName());
        }
    }
}

// Main class to demonstrate functionality
public class Main {
    public static void main(String[] args) {
        // Create instances of animals
        Animal dog = new Dog();
        Animal cat = new Cat();
        Animal bird = new Bird();

        // Create a Zoo instance
        Zoo zoo = new Zoo();
    }
}
```

```

// Add animals to the zoo

zoo.addAnimal(dog);

zoo.addAnimal(cat);

zoo.addAnimal(bird);


// Display the sounds of all animals

zoo.displaySounds();


// Display the animals that can fly

zoo.displayFlyers();
}
}

```

Questions 2

You are hired to design a **payment processing system** for an e-commerce application. The system should support multiple payment methods such as **CreditCard**, **PayPal**, and **BankTransfer**. Each payment method has its own way of processing a payment. The system should be flexible enough to allow the addition of new payment methods in the future without significant changes.

Requirements:

1. Create an abstract class `PaymentMethod` with:
 - An abstract method `processPayment(double amount)`.
2. Implement the following concrete subclasses:
 - **CreditCard**: Print "Processing \$amount payment through Credit Card."
 - **PayPal**: Print "Processing \$amount payment through PayPal."
 - **BankTransfer**: Print "Processing \$amount payment through Bank Transfer."
3. Create a `PaymentProcessor` class that:
 - Contains a method `process(PaymentMethod paymentMethod, double amount)` that takes any payment method and processes the payment.
4. Write a main program to:
 - Instantiate objects of `CreditCard`, `PayPal`, and `BankTransfer`.

- Use the `PaymentProcessor` to process payments using these payment methods.

Solution

```
// Abstract class
abstract class PaymentMethod {
    public abstract void processPayment(double amount);
}

// Subclass for CreditCard
class CreditCard extends PaymentMethod {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing $" + amount + " payment through Credit
Card.");
    }
}

// Subclass for PayPal
class PayPal extends PaymentMethod {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing $" + amount + " payment through
PayPal.");
    }
}

// Subclass for BankTransfer
class BankTransfer extends PaymentMethod {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing $" + amount + " payment through Bank
Transfer.");
    }
}

// PaymentProcessor class
class PaymentProcessor {
    public void process(PaymentMethod paymentMethod, double amount) {
        paymentMethod.processPayment(amount);
    }
}

// Main class
public class Main {
    public static void main(String[] args) {
        // Create payment method instances
        PaymentMethod creditCard = new CreditCard();
        PaymentMethod paypal = new PayPal();
        PaymentMethod bankTransfer = new BankTransfer();
    }
}
```



```

        // Create a PaymentProcessor
        PaymentProcessor processor = new PaymentProcessor();

        // Process payments
        processor.process(creditCard, 100.50);
        processor.process(payPal, 200.75);
        processor.process(bankTransfer, 500.00);
    }
}

```

Questions 3

You are hired by a logistics company to develop a system to manage different types of vehicles. Each vehicle can calculate its maximum range based on its fuel capacity and fuel efficiency. The company has three types of vehicles: **Car**, **Truck**, and **Motorcycle**. Each type has unique characteristics for calculating the range.

Requirements:

1. Create a parent class `Vehicle` with the following:
 - An abstract method `calculateRange()` that computes the maximum range of the vehicle.
 - A common property `fuelCapacity` to store the amount of fuel the vehicle can hold.
2. Implement the following subclasses:
 - **Car**: Consumes 10 units of fuel per 1 kilometer.
 - **Truck**: Consumes 5 units of fuel per 1 kilometer.
 - **Motorcycle**: Consumes 15 units of fuel per 1 kilometer.
3. Create a `Logistics` class to manage a list of vehicles. It should include:
 - A method `addVehicle(Vehicle vehicle)` to add a vehicle to the list.
 - A method `displayRanges()` to calculate and display the range of each vehicle.
4. In the main program:
 - Create instances of `Car`, `Truck`, and `Motorcycle` with different fuel capacities.
 - Add these vehicles to the `Logistics` object.
 - Call `displayRanges()` to demonstrate polymorphism.

Solution

```

// Abstract parent class
abstract class Vehicle {
    protected double fuelCapacity;
}

```

```
// Constructor

public Vehicle(double fuelCapacity) {
    this.fuelCapacity = fuelCapacity;
}

// Abstract method to calculate range
public abstract double calculateRange();
}

// Subclass: Car
class Car extends Vehicle {
    public Car(double fuelCapacity) {
        super(fuelCapacity);
    }

    @Override
    public double calculateRange() {
        return fuelCapacity * 10; // 10 units per kilometer
    }
}

// Subclass: Truck
class Truck extends Vehicle {
    public Truck(double fuelCapacity) {
        super(fuelCapacity);
    }

    @Override
    public double calculateRange() {
```

```

        return fuelCapacity * 5; // 5 units per kilometer
    }
}

// Subclass: Motorcycle
class Motorcycle extends Vehicle {
    public Motorcycle(double fuelCapacity) {
        super(fuelCapacity);
    }

    @Override
    public double calculateRange() {
        return fuelCapacity * 15; // 15 units per kilometer
    }
}

// Logistics class
class Logistics {
    private List<Vehicle> vehicles;

    // Constructor
    public Logistics() {
        this.vehicles = new ArrayList<>();
    }

    // Method to add a vehicle
    public void addVehicle(Vehicle vehicle) {
        vehicles.add(vehicle);
    }
}

```

```

// Method to display ranges
public void displayRanges() {
    for (Vehicle vehicle : vehicles) {
        System.out.println(vehicle.getClass().getSimpleName() + " Range: " + vehicle.calculateRange() + "
km");
    }
}

}

// Main class
public class Main {

    public static void main(String[] args) {

        // Create vehicles

        Vehicle car = new Car(50);      // 50 liters
        Vehicle truck = new Truck(100); // 100 liters
        Vehicle motorcycle = new Motorcycle(20); // 20 liters


        // Manage vehicles in logistics
        Logistics logistics = new Logistics();
        logistics.addVehicle(car);
        logistics.addVehicle(truck);
        logistics.addVehicle(motorcycle);


        // Display ranges
        logistics.displayRanges();
    }
}

```

Explanation of Solution:

1. **Polymorphism:**

- The abstract class `Vehicle` defines a blueprint for `calculateRange()`.
- Each subclass (`Car`, `Truck`, `Motorcycle`) overrides `calculateRange()` to implement specific behavior.

2. **Code Reusability:**

- The `Logistics` class uses a list of `Vehicle` objects and calls `calculateRange()` polymorphically, without needing to know the specific type of vehicle.

Questions 4

A tech company wants to build a **Payment Processing System** that supports multiple payment methods such as **Credit Card**, **Debit Card**, and **Digital Wallet**. To ensure flexibility and security, the company has the following requirements:

1. **Encapsulation:**

- Each payment method should encapsulate sensitive details like card number or wallet ID.
- These details must only be accessible through specific methods.

2. **Interface:**

- Define a common interface `Payment` with a method `processPayment(double amount)` to ensure all payment methods follow the same structure.
- Each payment method will have its own implementation of `processPayment`.

3. Implement three payment methods:

- **CreditCardPayment:** Requires a card number and a CVV.
- **DebitCardPayment:** Requires a card number and a PIN.
- **DigitalWalletPayment:** Requires a wallet ID and a password.

4. Create a `PaymentProcessor` class that accepts any payment method and processes a payment.

5. Implement a `Main` program to:

- Create instances of the payment methods.
- Process payments using the `PaymentProcessor`.

Solution

```
// Payment interface
```

```
interface Payment
```

```
{
```

```
    void processPayment(double amount);
```

```
}
```

```
// Credit Card Payment class
class CreditCardPayment implements Payment
{
    private String cardNumber;
    private String cvv;

    // Constructor
    public CreditCardPayment(String cardNumber, String cvv)
    {
        this.cardNumber = cardNumber;
        this.cvv = cvv;
    }

    // Getter methods (encapsulated access to sensitive data)
    public String getCardNumber()
    {
        return cardNumber;
    }

    public String getCvv()
    {
        return cvv;
    }

    @Override
    public void processPayment(double amount)
    {
        System.out.println("Processing credit card payment of $" + amount);
    }
}
```

```
}  
}
```

```
// Debit Card Payment class
```

```
class DebitCardPayment implements Payment
```

```
{
```

```
    private String cardNumber;
```

```
    private String pin;
```

```
    // Constructor
```

```
    public DebitCardPayment(String cardNumber, String pin)
```

```
{
```

```
    this.cardNumber = cardNumber;
```

```
    this.pin = pin;
```

```
}
```

```
    // Getter methods (encapsulated access to sensitive data)
```

```
    public String getCardNumber()
```

```
{
```

```
    return cardNumber;
```

```
}
```

```
    public String getPin()
```

```
{
```

```
    return pin;
```

```
}
```

```
@Override
```

```
    public void processPayment(double amount)
    {
        System.out.println("Processing debit card payment of $" + amount);
    }
}
```

// Digital Wallet Payment class

class DigitalWalletPayment implements Payment

```
{
    private String walletId;
    private String password;
```

// Constructor

public DigitalWalletPayment(String walletId, String password)

```
{
    this.walletId = walletId;
    this.password = password;
}
```

// Getter methods (encapsulated access to sensitive data)

public String getWalletId()

```
{
    return walletId;
}
```

public String getPassword()

```
{
    return password;
}
```



```

    }

    @Override
    public void processPayment(double amount)
    {
        System.out.println("Processing digital wallet payment of $" + amount);
    }
}

// Payment Processor class
class PaymentProcessor
{
    public void process(Payment payment, double amount)
    {
        payment.processPayment(amount);
    }
}

// Main class
public class Main
{
    public static void main(String[] args)
    {
        // Create payment methods
        Payment creditCard = new CreditCardPayment("1234567890123456", "123");
        Payment debitCard = new DebitCardPayment("9876543210987654", "4567");
        Payment digitalWallet = new DigitalWalletPayment("wallet123", "password");
    }
}

```

```
// Create payment processor
PaymentProcessor processor = new PaymentProcessor();

// Process payments
processor.process(creditCard, 100.50);
processor.process(debitCard, 200.75);
processor.process(digitalWallet, 300.00);
}
}
```

Questions 5

A travel agency provides an online booking system for customers to reserve seats on flights. Due to increasing demand, they need to ensure their booking system can handle errors gracefully and enforce business rules.

Requirements:

1. The program should:
 - Accept the following inputs:
 - `availableSeats` (number of available seats on the flight)
 - `requestedSeats` (number of seats the customer wants to book)
 - Perform the following checks:
 - If the requested seats are less than or equal to 0, throw an `IllegalArgumentException`.
 - If the requested seats exceed available seats, throw a `RuntimeException` with a message "Not enough seats available".
 - If available seats are zero, throw a `BookingException` (a custom exception) with the message "Flight is fully booked".
2. Display appropriate success or error messages based on the outcomes.
3. Use a `try-catch` block to handle all exceptions.

Questions

1. Implement the booking logic using the above scenario.

2. Explain why custom exceptions like `BookingException` might be useful in real-world applications.
3. Modify the program to allow retrying the booking process if an exception occurs.

Key Takeaways

1. **Exception Handling:** Ensures the program can gracefully manage errors without crashing.
2. **Custom Exceptions:** Provide clarity and domain-specific error handling.
3. **Validation with `if` Statements:** Prevent invalid states before executing further logic.

Solution

```
// Custom exception class
```

```
class BookingException extends Exception {  
    public BookingException(String message) {  
        super(message);  
    }  
}
```

```
public class FlightBookingSystem {
```

```
    public static void main(String[] args) {
```

```
        int availableSeats = 5; // Example available seats
```

```
        // Simulate user inputs
```

```
        int[] requestedSeatsList = {3, 0, 6, 2};
```

```
        for (int requestedSeats : requestedSeatsList) {
```

```
            System.out.println("Trying to book " + requestedSeats + " seats...");
```

```
            try {
```

```

        bookSeats(availableSeats, requestedSeats);
        availableSeats -= requestedSeats;
        System.out.println("Seats booked successfully. Remaining seats: " + availableSeats);
    } catch (IllegalArgumentException e) {
        System.out.println("Error: " + e.getMessage());
    } catch (RuntimeException e) {
        System.out.println("Error: " + e.getMessage());
    } catch (BookingException e) {
        System.out.println("Error: " + e.getMessage());
        break; // No more bookings allowed as the flight is fully booked
    } finally {
        System.out.println("Booking attempt finished.\n");
    }
}
}

```

```

public static void bookSeats(int availableSeats, int requestedSeats) throws BookingException {
    if (requestedSeats <= 0) {
        throw new IllegalArgumentException("Requested seats must be greater than 0.");
    }
    if (availableSeats == 0) {
        throw new BookingException("Flight is fully booked.");
    }
    if (requestedSeats > availableSeats) {
        throw new RuntimeException("Not enough seats available.");
    }
}
}

```


Question 6

A software company has developed an application that manages user accounts. The application has a feature to transfer funds between accounts. However, the system must handle several situations, including invalid input, insufficient balance, and maximum transfer limits.

Task:

Write a Java program to simulate the fund transfer functionality. The program should:

1. Accept inputs for:
 - Sender's account balance
 - Transfer amount
 2. Perform the following checks:
 - Ensure the transfer amount is positive. If not, throw an `IllegalArgumentException`.
 - Ensure the sender has enough balance for the transfer. If not, throw an `ArithmeticException`.
 - Ensure the transfer amount does not exceed \$10,000. If it does, throw a `SecurityException`.
 3. If all conditions are met, deduct the amount from the sender's balance and display the updated balance.
 4. Use a try-catch block to handle exceptions and display appropriate error messages.
-

Questions

1. Implement the `fundTransfer` method using the above scenario.
2. Explain the role of exceptions in ensuring robust error handling.
3. Extend the program to log the exceptions to a text file for audit purposes.

Explanation

1. **Validation and Exceptions:** The program uses `if` statements to validate the inputs and throws specific exceptions for invalid cases.
2. **Try-Catch Block:** Handles exceptions and provides user-friendly error messages.
3. **Reusability:** The `transferFunds` method encapsulates the business logic for fund transfer and can be reused in other contexts.

Solution

```
import java.util.Scanner;

public class FundTransfer
{

    public static void main(String[] args)
    {

        Scanner scanner = new Scanner(System.in);

        try
        {
            // Input: Sender's account balance
            System.out.print("Enter sender's account balance: ");
            double accountBalance = scanner.nextDouble();

            // Input: Transfer amount
            System.out.print("Enter transfer amount: ");
            double transferAmount = scanner.nextDouble();

            // Perform the transfer
            double updatedBalance = transferFunds(accountBalance, transferAmount);

            // If no exceptions are thrown, display the updated balance
            System.out.println("Transfer successful! Updated balance: $" + updatedBalance);

        }
        catch (IllegalArgumentException e)
```

```

    {
        System.out.println("Error: " + e.getMessage());
    }
    catch (ArithmeticException e)
    {
        System.out.println("Error: " + e.getMessage());
    }
    catch (SecurityException e)
    {
        System.out.println("Error: " + e.getMessage());
    }
    finally
    {
        System.out.println("Transaction process completed.");
    }

    scanner.close();
}

/**
 * Method to handle fund transfer logic.
 *
 * @param accountBalance The sender's current account balance.
 * @param transferAmount The amount to transfer.
 * @return The updated account balance after a successful transfer.
 * @throws IllegalArgumentException if the transfer amount is not positive.
 * @throws ArithmeticException    if the account balance is insufficient.
 * @throws SecurityException       if the transfer amount exceeds the limit.

```



```

*/
public static double transferFunds(double accountBalance, double transferAmount)
{
    // Check if the transfer amount is positive
    if (transferAmount <= 0)
    {
        throw new IllegalArgumentException("Transfer amount must be greater than zero.");
    }

    // Check if the transfer amount exceeds the account balance
    if (transferAmount > accountBalance)
    {
        throw new ArithmeticException("Insufficient balance for the transfer.");
    }

    // Check if the transfer amount exceeds $10,000
    if (transferAmount > 10000)
    {
        throw new SecurityException("Transfer amount exceeds the maximum limit of $10,000.");
    }

    // Deduct the transfer amount from the account balance
    return accountBalance - transferAmount;
}
}

```

Question 7

You are tasked with creating a **Student Registration System** using Java Swing for the GUI and a database (e.g., MySQL or SQLite) to store student information. The system should allow the user

to input student details (Name, Age, Course, Email) and save them into the database. Additionally, the system should have a feature to display all registered students in a `JTable`.

Requirements

1. GUI Components:

- A form with the following fields:
 - `TextField` for Name, Age, and Email.
 - A `ComboBox` for selecting a Course (e.g., "Computer Science", "Mathematics", "Physics").
- A `Button` to register the student.
- A `Button` to display all registered students.

2. Database:

- A table named `students` with the following columns:
`id (INT PRIMARY KEY AUTO_INCREMENT), name (VARCHAR), age (INT), course (VARCHAR), email (VARCHAR).`

3. Features:

- Validate the form (e.g., ensure no fields are empty, Age is a number, Email is in a valid format).
- Save the student data into the database when the **Register** button is clicked.
- Display all registered students in a `JTable` when the **Show Students** button is clicked.

Expected Functionality:

- **Registration:** Validates the form, saves the data to the database, and shows a success message.
- **Display Students:** Populates the `JTable` with all registered students from the database.
- **Error Handling:** Shows meaningful error messages when database connectivity fails or invalid input is provided.

Solution

```
import javax.swing.*;

import javax.swing.table.DefaultTableModel;

import java.awt.*;

import java.awt.event.ActionEvent;

import java.awt.event.ActionListener;
```

```

import java.sql.*;

public class StudentRegistration
{
    private static final String DB_URL = "jdbc:mysql://localhost:3306/studentdb";
    private static final String DB_USER = "root";
    private static final String DB_PASSWORD = "password";

    public static void main(String[] args)
    {
        SwingUtilities.invokeLater(() -> new StudentRegistration().createAndShowGUI());
    }

    private void createAndShowGUI()
    {
        JFrame frame = new JFrame("Student Registration");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(500, 500);

        JPanel panel = new JPanel(new GridLayout(6, 2, 10, 10));

        // Form Components
        JTextField nameField = new JTextField();
        JTextField ageField = new JTextField();
        JTextField emailField = new JTextField();
        JComboBox<String> courseBox = new JComboBox<>(new String[]{"Computer Science",
"Mathematics", "Physics"});

        JButton registerButton = new JButton("Register");
        JButton showButton = new JButton("Show Students");
    }
}

```

```

panel.add(new JLabel("Name:"));
panel.add(nameField);
panel.add(new JLabel("Age:"));
panel.add(ageField);
panel.add(new JLabel("Email:"));
panel.add(emailField);
panel.add(new JLabel("Course:"));
panel.add(courseBox);
panel.add(registerButton);
panel.add(showButton);

frame.add(panel, BorderLayout.NORTH);

// Table for displaying students
JTable table = new JTable(new DefaultTableModel(new String[]{"ID", "Name", "Age",
"Course", "Email"}, 0));
frame.add(new JScrollPane(table), BorderLayout.CENTER);

frame.setVisible(true);

// Button Listeners
registerButton.addActionListener(new ActionListener()
{
    @Override
    public void actionPerformed(ActionEvent e)
    {
        // TODO: Implement registration logic
    }
}

```

```

});

showButton.addActionListener(new ActionListener()
{
    @Override
    public void actionPerformed(ActionEvent e)
    {
        // TODO: Implement logic to fetch and display students
    }
});
}

// Method to connect to the database
private Connection connectToDatabase()
{
    try
    {
        return DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
    } catch (SQLException e)
    {
        JOptionPane.showMessageDialog(null, "Database connection failed: " + e.getMessage());
        return null;
    }
}
}

```

Question 8

You are tasked with creating a **Hotel Reservation System** using Java Swing for the user interface and a database (e.g., MySQL or SQLite) for storing reservation details. The system allows users to book a room and view existing reservations.

Requirements

1. GUI Components:

- A form with the following fields:
 - `TextField` for Guest Name, Email, and Number of Guests.
 - `ComboBox` for Room Type (e.g., "Single", "Double", "Suite").
 - `Spinner` for Check-in and Check-out dates.
- A `Button` to make a reservation.
- A `Button` to display all reservations.

2. Database:

- A table named `reservations` with the following schema:

```
sql
Copy code
id (INT PRIMARY KEY AUTO_INCREMENT)
guest_name (VARCHAR)
email (VARCHAR)
room_type (VARCHAR)
number_of_guests (INT)
check_in_date (DATE)
check_out_date (DATE)
```

3. Features:

- Validate the form to ensure no fields are empty, email is valid, and the check-out date is after the check-in date.
- Save reservation details to the database when the **Make Reservation** button is clicked.
- Display all reservations in a `JTable` when the **Show Reservations** button is clicked.

Solution

```
import javax.swing.*;

import javax.swing.table.DefaultTableModel;

import java.awt.*;

import java.awt.event.ActionEvent;

import java.awt.event.ActionListener;

import java.sql.*;

import java.util.Date;
```

```
public class HotelReservation
```

```

{
    private static final String DB_URL = "jdbc:mysql://localhost:3306/hoteldb";
    private static final String DB_USER = "root";
    private static final String DB_PASSWORD = "password";

    public static void main(String[] args)
    {
        SwingUtilities.invokeLater(() -> new HotelReservation().createAndShowGUI());
    }

    private void createAndShowGUI()
    {
        JFrame frame = new JFrame("Hotel Reservation");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(600, 600);

        JPanel formPanel = new JPanel(new GridLayout(7, 2, 10, 10));
        JTextField nameField = new JTextField();
        JTextField emailField = new JTextField();
        JTextField guestsField = new JTextField();
        JComboBox<String> roomTypeBox = new JComboBox<>(new String[]{"Single", "Double", "Suite"});
        JSpinner checkInSpinner = new JSpinner(new SpinnerDateModel());
        JSpinner checkOutSpinner = new JSpinner(new SpinnerDateModel());
        JButton reserveButton = new JButton("Make Reservation");
        JButton showButton = new JButton("Show Reservations");

        formPanel.add(new JLabel("Guest Name:"));
        formPanel.add(nameField);

```

```

formPanel.add(new JLabel("Email:"));
formPanel.add(emailField);
formPanel.add(new JLabel("Number of Guests:"));
formPanel.add(guestsField);
formPanel.add(new JLabel("Room Type:"));
formPanel.add(roomTypeBox);
formPanel.add(new JLabel("Check-in Date:"));
formPanel.add(checkInSpinner);
formPanel.add(new JLabel("Check-out Date:"));
formPanel.add(checkOutSpinner);
formPanel.add(reserveButton);
formPanel.add(showButton);

frame.add(formPanel, BorderLayout.NORTH);

// Table for reservations
JTable table = new JTable(new DefaultTableModel(new String[]{"ID", "Guest Name",
"Email", "Room Type", "Guests", "Check-in", "Check-out"}, 0));
frame.add(new JScrollPane(table), BorderLayout.CENTER);

frame.setVisible(true);

// Button Actions
reserveButton.addActionListener(new ActionListener()
{
    @Override
    public void actionPerformed(ActionEvent e)
    {
        // TODO: Implement reservation logic
    }
}

```



```

    }
});

showButton.addActionListener(new ActionListener()
{
    @Override
    public void actionPerformed(ActionEvent e)
    {
        // TODO: Implement logic to display reservations
    }
});
}

private Connection connectToDatabase()
{
    try
    {
        return DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
    }
    catch (SQLException e)
    {
        JOptionPane.showMessageDialog(null, "Database connection failed: " + e.getMessage());
        return null;
    }
}
}

```

Question 9

You are tasked with creating a **Payroll Management System** using Java Swing for the user interface and Microsoft SQL Server for the database. The system allows HR personnel to manage employee salary details, including adding new employee records, calculating net pay, and viewing all employee payroll records.

Requirements

1. GUI Components:

- A form with the following fields for employee details:
 - `TextField` for Employee Name, Employee ID, and Basic Salary.
 - `TextField` for Allowances and Deductions.
- A `Button` to calculate and save the payroll record.
- A `Button` to display all payroll records in a table.

2. Database:

- A table named `Payroll` with the following schema:

```
sql
Copy code
id (INT PRIMARY KEY AUTO_INCREMENT),
employee_id (VARCHAR),
employee_name (VARCHAR),
basic_salary (FLOAT),
allowances (FLOAT),
deductions (FLOAT),
net_pay (FLOAT)
```

3. Features:

- Calculate **Net Pay** as:
$$\text{Net Pay} = \text{Basic Salary} + \text{Allowances} - \text{Deductions}.$$
- Validate that all fields are filled and numeric fields contain valid numbers.
- Save the payroll details into the `Payroll` table when the **Save Payroll** button is clicked.
- Display all payroll records in a `JTable` when the **View Payroll** button is clicked.

Expected Outputs

- **Save Payroll:** Successfully validates input, calculates Net Pay, and saves the record to the database. Displays a success message.
- **View Payroll:** Displays all existing payroll records in the `JTable`. If no records exist, the table remains empty.
- **Error Handling:** Handles database connection errors or invalid input gracefully.

Solution

```
import javax.swing.*;

import javax.swing.table.DefaultTableModel;

import java.awt.*;

import java.awt.event.ActionEvent;

import java.awt.event.ActionListener;

import java.sql.*;


public class PayrollSystem
{
    private static final String DB_URL =
"jdbc:sqlserver://localhost:1433;databaseName=PayrollDB";

    private static final String DB_USER = "your_username";

    private static final String DB_PASSWORD = "your_password";


    public static void main(String[] args)
    {
        SwingUtilities.invokeLater(() -> new PayrollSystem().createAndShowGUI());
    }


    private void createAndShowGUI()
    {
        JFrame frame = new JFrame("Payroll Management System");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(600, 600);


        JPanel formPanel = new JPanel(new GridLayout(6, 2, 10, 10));

        JTextField empIDField = new JTextField();

        JTextField empNameField = new JTextField();
```

```

    JTextField basicSalaryField = new JTextField();
    JTextField allowancesField = new JTextField();
    JTextField deductionsField = new JTextField();
    JButton saveButton = new JButton("Save Payroll");
    JButton viewButton = new JButton("View Payroll");

    formPanel.add(new JLabel("Employee ID:"));
    formPanel.add(empIDField);
    formPanel.add(new JLabel("Employee Name:"));
    formPanel.add(empNameField);
    formPanel.add(new JLabel("Basic Salary:"));
    formPanel.add(basicSalaryField);
    formPanel.add(new JLabel("Allowances:"));
    formPanel.add(allowancesField);
    formPanel.add(new JLabel("Deductions:"));
    formPanel.add(deductionsField);
    formPanel.add(saveButton);
    formPanel.add(viewButton);

    frame.add(formPanel, BorderLayout.NORTH);

    JTable table = new JTable(new DefaultTableModel(new String[]{"ID", "Employee ID",
    "Name", "Basic Salary", "Allowances", "Deductions", "Net Pay"}, 0));
    frame.add(new JScrollPane(table), BorderLayout.CENTER);

    frame.setVisible(true);

    // Button Listeners
    saveButton.addActionListener(new ActionListener()

```

```

{
    @Override
    public void actionPerformed(ActionEvent e)
    {
        // TODO: Implement logic to calculate net pay and save record
    }
});

viewButton.addActionListener(new ActionListener()
{
    @Override
    public void actionPerformed(ActionEvent e)
    {
        // TODO: Implement logic to fetch and display records
    }
});
}

// Database connection method
private Connection connectToDatabase()
{
    try
    {
        return DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
    }
    catch (SQLException e)
    {
        JOptionPane.showMessageDialog(null, "Database connection failed: " + e.getMessage());
    }
}

```

```
        return null;
    }
}
}
```

Question 10

You are tasked with creating a **Sales Inventory Management System** using Java Swing for the user interface and Microsoft SQL Server for the database. The system will allow users to add products, update stock levels, record sales, and view inventory details.

Requirements

1. GUI Components:

- A form for **Product Management** with the following fields:
 - `TextField` for Product ID, Product Name, Price per Unit, and Quantity in Stock.
 - `Button` for **Add Product** and **Update Stock**.
- A form for **Sales Management** with the following fields:
 - `TextField` for Product ID and Quantity Sold.
 - `Button` to **Record Sale**.
- A `Button` to **View Inventory**, which displays all product details in a `JTable`.

2. Database:

- A table named `Products` with the following schema:

```
product_id (VARCHAR PRIMARY KEY),
product_name (VARCHAR),
price_per_unit (FLOAT),
quantity_in_stock (INT)
```

- A table named `Sales` with the following schema:

```
sale_id (INT PRIMARY KEY AUTO_INCREMENT),
product_id (VARCHAR),
quantity_sold (INT),
sale_date (DATETIME)
```

3. Features:

- **Add Product:** Adds a new product to the `Products` table.
- **Update Stock:** Updates the quantity of an existing product in the `Products` table.
- **Record Sale:** Validates the stock availability and inserts the sale details into the `Sales` table, while updating the `Products` table to decrease the stock level.

- **View Inventory:** Displays all product details from the `Products` table in a `JTable`.

```
import javax.swing.*;

import javax.swing.table.DefaultTableModel;

import java.awt.*;

import java.awt.event.ActionEvent;

import java.awt.event.ActionListener;

import java.sql.*;

public class SalesInventorySystem
{
    private static final String DB_URL =
"jdbc:sqlserver://localhost:1433;databaseName=SalesInventoryDB";

    private static final String DB_USER = "your_username";

    private static final String DB_PASSWORD = "your_password";

    public static void main(String[] args)
    {
        SwingUtilities.invokeLater(() -> new SalesInventorySystem().createAndShowGUI());
    }

    private void createAndShowGUI()
    {
        JFrame frame = new JFrame("Sales Inventory Management System");

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        frame.setSize(800, 600);

        // Product Management Panel
```

```
JPanel productPanel = new JPanel(new GridLayout(5, 2, 10, 10));
JTextField prodIDField = new JTextField();
JTextField prodNameField = new JTextField();
JTextField priceField = new JTextField();
JTextField stockField = new JTextField();
JButton addProductButton = new JButton("Add Product");
JButton updateStockButton = new JButton("Update Stock");
```

```
productPanel.add(new JLabel("Product ID:"));
productPanel.add(prodIDField);
productPanel.add(new JLabel("Product Name:"));
productPanel.add(prodNameField);
productPanel.add(new JLabel("Price per Unit:"));
productPanel.add(priceField);
productPanel.add(new JLabel("Quantity in Stock:"));
productPanel.add(stockField);
productPanel.add(addProductButton);
productPanel.add(updateStockButton);
```

```
// Sales Management Panel
```

```
JPanel salesPanel = new JPanel(new GridLayout(3, 2, 10, 10));
JTextField salesProdIDField = new JTextField();
JTextField quantitySoldField = new JTextField();
JButton recordSaleButton = new JButton("Record Sale");
```

```
salesPanel.add(new JLabel("Product ID:"));
salesPanel.add(salesProdIDField);
salesPanel.add(new JLabel("Quantity Sold:"));
```



```
salesPanel.add(quantitySoldField);
salesPanel.add(recordSaleButton);

// Inventory Table
JTable inventoryTable = new JTable(new DefaultTableModel(new String[]{"Product ID",
"Name", "Price", "Stock"}, 0));
JButton viewInventoryButton = new JButton("View Inventory");

// Layout
frame.setLayout(new BorderLayout());
frame.add(productPanel, BorderLayout.NORTH);
frame.add(salesPanel, BorderLayout.CENTER);
frame.add(new JScrollPane(inventoryTable), BorderLayout.SOUTH);
frame.add(viewInventoryButton, BorderLayout.EAST);

frame.setVisible(true);

// Button Listeners
addProductButton.addActionListener(new ActionListener()
{
    @Override
    public void actionPerformed(ActionEvent e)
    {
        // TODO: Implement Add Product logic
    }
});

updateStockButton.addActionListener(new ActionListener()
{
    {
```

```

@Override
public void actionPerformed(ActionEvent e)
{
    // TODO: Implement Update Stock logic
}
});

recordSaleButton.addActionListener(new ActionListener()
{
    @Override
    public void actionPerformed(ActionEvent e)
    {
        // TODO: Implement Record Sale logic
    }
});

viewInventoryButton.addActionListener(new ActionListener()
{
    @Override
    public void actionPerformed(ActionEvent e)
    {
        // TODO: Implement View Inventory logic
    }
});
}

private Connection connectToDatabase()
{

```

```
try
{
    return DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
}
catch (SQLException e)
{
    JOptionPane.showMessageDialog(null, "Database connection failed: " + e.getMessage());
    return null;
}
}
```